

API SECURITY RISK ANALYSIS REPORT

Target API : <https://jsonplaceholder.typicode.com>

Prepared By : Adithyan.V

Date : June 03, 2026

1. Introduction & Objectives

Application Programming Interfaces (APIs) act as the digital pipelines that allow different software programs to communicate and share data. If these pathways are left unprotected, unauthorized entities can observe, harvest, or alter internal information. The primary objective of this assessment is to identify logical vulnerabilities in a public testing system using safe, non-exploitative methods, aligning with industry standards like the OWASP framework. This evaluation translates technical operational flaws into plain business risks to guide engineering team remediation efforts.

2. Security Methodology

Following strict ethical and legal compliance guidelines, this assessment utilized a passive, read-only testing methodology. All observations were made by evaluating exposed endpoints, mapping routing structures, and inspecting background response headers using Postman. No vulnerability exploitation, administrative bypass configurations, or Denial of Service (DoS) flooding techniques were attempted.

3. Identified Vulnerabilities

3.1 Insecure Direct Object Reference (IDOR)

Severity: HIGH

Description: The application allows access to individual user records sequentially (via paths like /users/1, /users/2, and /users/3) simply by modifying the ID number in the web address string. The system fails to check whether the requesting party has authorization to access those specific records.

Impact: In a live business environment, a malicious actor can automate an address-scanning tool to cycle through sequential numbers and download the entire private user account directory.

Remediation: Implement server-side identity verification checks. Ensure that the application validates whether the security token of the logged-in client matches the ownership rules of the specific record being requested before sending data.

← → ↻ 🏠 jsonplaceholder.typicode.com/users/1

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB 403 Forbidden

JSON Raw Data Headers

Save Copy Collapse All Expand All Filter JSON

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  },
  "phone": "1-770-736-8031 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net",
    "bs": "harness real-time e-markets"
  }
}
```

← → ↻ 🏠 jsonplaceholder.typicode.com/users/2

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB 403 Forbidden

JSON Raw Data Headers

Save Copy Collapse All Expand All Filter JSON

```
{
  "id": 2,
  "name": "Ervin Howell",
  "username": "Antonette",
  "email": "Shanna@melissa.tv",
  "address": {
    "street": "Victor Plains",
    "suite": "Suite 879",
    "city": "Wisokyburgh",
    "zipcode": "90566-7771",
    "geo": {
      "lat": "-43.9509",
      "lng": "-34.4618"
    }
  },
  "phone": "010-692-6593 x09125",
  "website": "anastasia.net",
  "company": {
    "name": "Deckow-Crist",
    "catchPhrase": "Proactive didactic contingency",
    "bs": "synergize scalable supply-chains"
  }
}
```

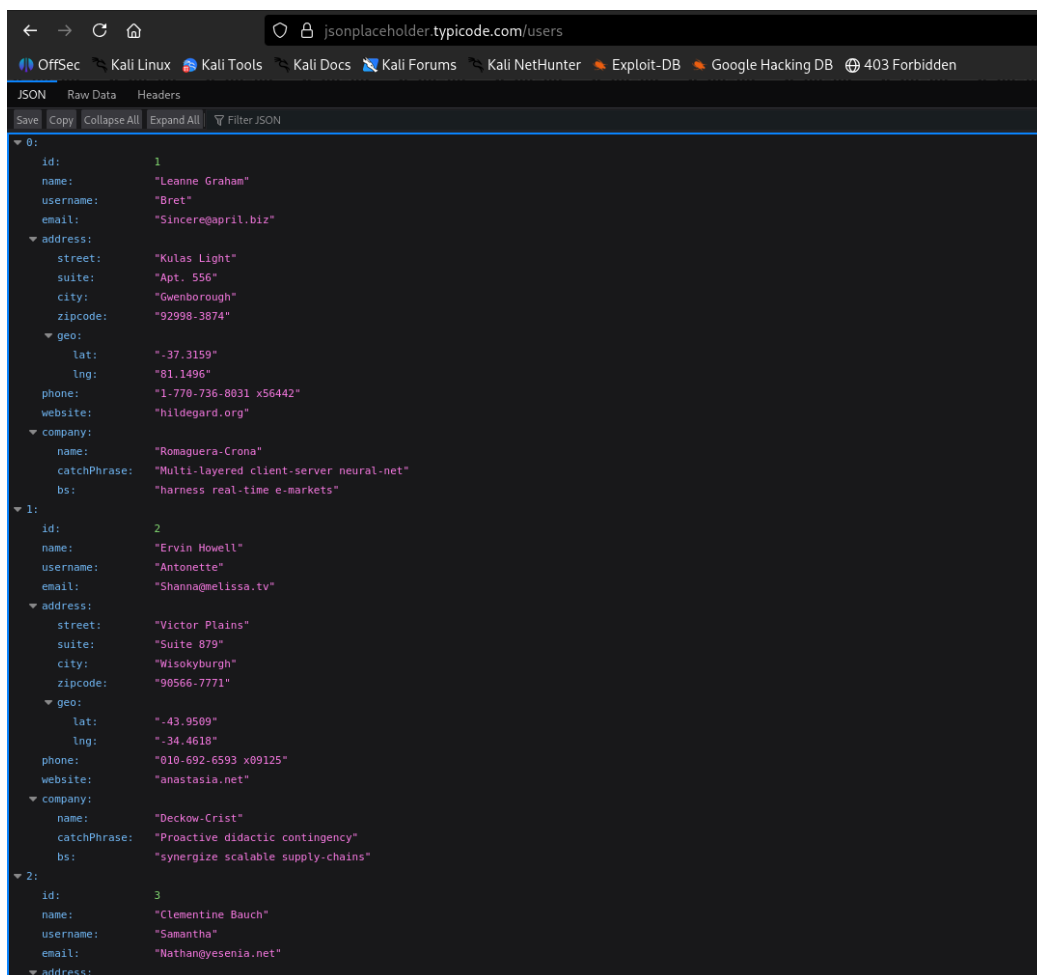
3.2 Mass Data Exposure

Severity: MEDIUM

Description: Accessing the root /users pathway prompts the application to return a complete list of every registered user directory in a single, combined response payload.

Impact: This design oversight allows external scrapers to capture the entire customer data map with a single request. Attackers can use harvested email or identity lists to perform targeted phishing scams or brute-force credential stuffing attacks against the company.

Remediation: Disable global collection endpoints for unauthenticated or public requests. Enforce discrete query parameters so users must search for specific accounts, and strictly filter out internal or non-public fields from public search results.



3.3 Improper Input Validation

Severity: MEDIUM

Description: When sending non-numeric data containing special symbols into a field meant strictly for integers (such as querying `posts?userid=2@rec`), the application backend fails to validate the data type. Instead of returning an error, it responds with a success status of 200 OK.

Impact: Accepting unverified or malformed input parameters indicates that the system's input processing layer is weak. Attackers routinely use poorly validated entry fields to pass injection scripts or malicious commands straight into backend storage systems.

Remediation: Enforce strict server-side type constraints. Configure the system routing logic to immediately block any request containing unexpected letters or special characters in a numeric field, returning an explicit 400 Bad Request error.

The screenshot shows a REST client interface with the following details:

- URL:** `https://jsonplaceholder.typicode.com/posts?userid=2@rec`
- Method:** GET
- Query Params:**

Key	Value	Description
userid	2@rec	
- Status:** 200 OK (811 ms, 7.98 KB)
- Body (JSON):**

```
[{"userId": 1, "id": 1, "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit", "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"}, {"userId": 1, "id": 2, "title": "qui est esse", "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"}, {"userId": 1, "id": 3, "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut", "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius odio et labore et velit aut"}]
```

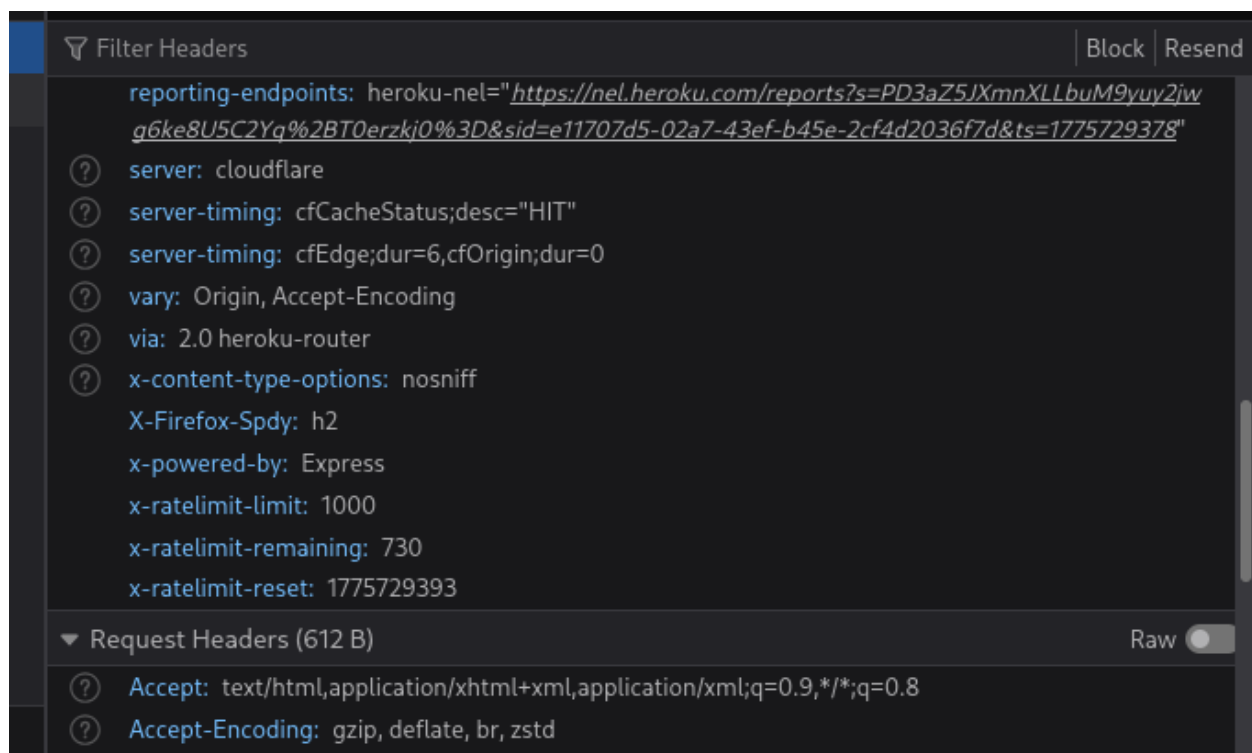
3.4 Permissive Rate Limiting & System Leak

Severity: LOW

Description: Analysis of the response headers shows that the active rate-limiting framework allows a highly permissive ceiling of 1,000 requests per user tracking window. Simultaneously, the response metadata openly names the specific backend hosting and development software platforms (Express and Heroku).

Impact: A loose limit of 1,000 requests allows automated extraction scripts to scrape substantial amounts of customer records before any automated defense trips. Furthermore, advertising the exact brands of your backend software provides attackers with a direct blueprint to target known software flaws.

Remediation: Lower the maximum request allowance on standard query endpoints to match normal human usage (e.g., 60 requests per minute). Configure the server environment to strip out identifying signatures, such as the x-powered-by header, before data reaches the public internet.



4. Summary Remediation Strategy

To systematically harden the application against the found gaps, the development team should apply defensive engineering adjustments in three clear phases:

1. Enforce Identity Checks: Never assume a user is allowed to view a record simply because they know the URL link. Validate account ownership on the backend for every data request.

2. Filter Entry Fields: Deploy validation filters to block special characters and wrong data formats from passing into the database engine.

3. Adjust Traffic Caps and Metadata: Drop the maximum request limits to prevent rapid data mining, and hide identifying brand signatures from background response headers.

5. Conclusion

In summary, the security evaluation highlighted notable vulnerabilities in the API's authentication and authorization frameworks. While these risks are currently contained within a testing environment, they represent high-severity entry points if replicated in a live environment. Enforcing strict access controls and regular security validation will effectively mitigate these risks and elevate the overall security posture of the API.